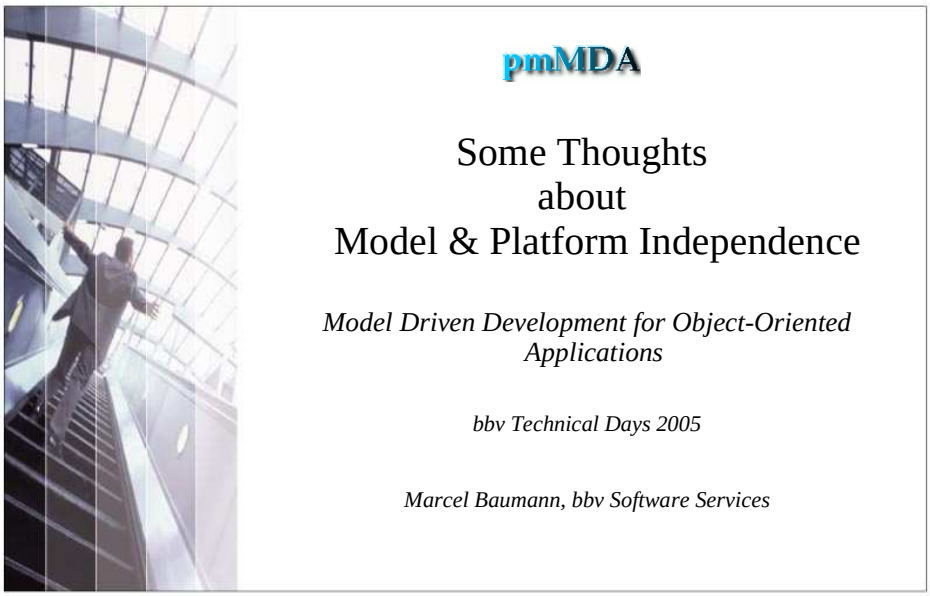


The *M*odel *D*riven *A*pproach – MDA - or the *M*odel *D*riven *D*esign - MDD - is an architectural concept promoting the usage of models. These models should be independent of the underlying middleware or programming languages. When we develop software, we need knowledge about two concerns.

- Domain aspects relevant to the software end-users
- Software technology aspects

The domain knowledge has nothing to do with software technology. The end-user of the software could not care less about technology. So only later should we decide if our application will be realized in a particular technology such as .NET, J2EE or CORBA.

This approach has clear advantages. During the lifetime of an application – five to fifteen years – the middleware technologies and programming languages change.

The *U*nified *M*odeling *L*anguage – UML - is the standard notation to develop such models. How do we model with UML to insure our models are platform independent?

This presentation provides some thoughts how UML models should be created. At the end of the presentation participants should have answers for

- What can I do with MDD and does it apply to my projects?
- What are trends in MDD?
- Which shortcuts and drawbacks were identified?

pmMDA

Contents

- Model independence (UML, MDA, MDD)
  - Datatypes, tagged values, stereotypes
  - Syntax versus semantic independence
  - Implementation in *pmMDA*
- Platform independence
  - SOA and web services – SOAP 1.x, locator -
  - Java, XML, C/C++
  - Open issues – authorization, authentication, transactions
  - Implementation in *pmMDA*

bbv behind things.

The following themes are discussed in the presentation.

- Model independence
  - Which mechanisms are provided in UML?
  - Are these mechanisms adequate or must the architect performs redundant work?
  - Which solution is provided in *pmMDA*?
- Platform independence
  - The current hype for platform independence is *S*ervice *O*riented *A*rchitecture – SOA -. The current silver bullet for SOA is *W*eb *S*ervices and the associated standards – WSDL, SOAP -.
    Some readers probably remember that CORBA had similar arguments.
  - Commercial companies have a system mix written in Java, C++, C and often in other languages – C#, COBOL, Delphi, PL-SQL -
  - Some hints are provided how services should be modeled and how they are mapped to the implementation language. A major problem is the integration of orthogonal configuration aspects such authentication, authorization, transaction, location within the models.
  - Which solutions should be provided in *pmMDA*?

This presentation shall present current facts about MDA/MDD and SOA. The facts were collected during projects done in insurance companies.

pmMDA

Axioms

- MDA/MDD is the modeling approach for complex systems
  - Standards are used *UML*, *XMI*, *MOF*, ...
- Platform independence is important
  - OS - UNIX, Windows, Mainframe -
  - Language – Java, C/C++, C#, VB -
  - Database – SQL Server, DB2, Oracle -
- Wizard and hand-crafted code cannot be maintained economically

bbv behind things.

Through MDD/MDA technologies, enabling validation and execution of models, we can clearly get a grasp of what the system is doing early on, and find flaws in the design that can be immediately corrected. The clear advantage is that it allows validation of the system early on, rather than later when it is too costly to find errors. MDD technology is still relatively new.

The hidden goal of MDD is to increase the abstraction level of your models. The models are no more polluted with platforms specific artifacts. Experience shows that more than 50% of the application classes are platform specific.

Model-driven development approaches and technologies in which development is centered round the manipulation of models raise the level of abstraction. It improve our abilities to develop complex systems.

Model Independence

- Datatypes
  - Language independent types, business model extensions
- Stereotypes
  - Mark UML items (often limited to one stereotype)
    e.g. data object, reference code, service
- Tagged values
  - key/value information
    e.g. bo-datatype, ojb-tablename

bbv behind things.

First you should realize that MDA actually means a number of distinctly different things to different people. Three camps exist.

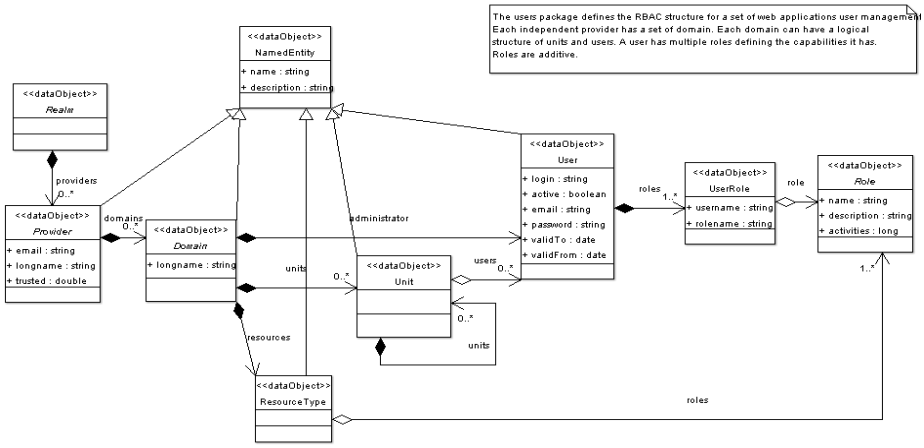
- *UML PIM* uses UML to build *P*latform *I*ndependent *M*odels which are transformed into *P*latform *S*pecific *M*odels – PSM -. These models are finally transformed into code.
- *MOF* do not use UML at all but define languages and transformations using the *O*bject *M*anagement *G*roup -OMG – *M*eta *O*bject *F*acility -MOF -.
- *Executable UML* builds a compiler for the UML and treats the UML as a programming language.

The MDD movement uses the term of *D*omain *S*pecific *L*anguage – DSL -. In general their supporters are not using OMG standards.

The approach discussed here is based on the UML PIM approach. The emphasis on frameworks in *pmMDA* has the potential to also support executable UML. Currently we are not supporting MOF due to its complexity. The underlying concept of *pmMDA* is to define a language workbench for a set of similar applications. UML is used as a notation because it is the industrial standard for software modelization.

UML does not provide mechanisms to describe complex domain entities with inherent information such as reference code values.

Model Independence



The above class diagram describes a simplified model for a *R*ole *B*ased *A*ccess *C*ontrol – RBAC – system. The classes and their properties are used to generate

- A persistence layer with the object to relation mapping rules and the database definition with integrity rules – DDL -.
- The value objects to transfer information between clients and servers in N tiers applications using either .NET, J2EE or CORBA.
- A simplified user interface to edit object graphs of the above class model.

Syntax versus Semantic

- Example Datatype
  - String => *Java*: java.lang.String
    => *C++*: std::string
  - But string/String is mapped to VARCHAR in datatype and language type: *must the architect specify multiple times the same information?*
  - Possible Approaches
    - Syntax: multiple definitions of the same information
    - Semantic: tool decides based on rules => no more transparent to the developer

pmMDA

pmMDA Approach

- Semantic approach is implemented
  - Tool infers derived information
  - System architect can overwrite the rules on a global basis
    - (+) flexible
    - (+) same rules for the whole project
    - (-) complex for the system architect

bbv behind things.

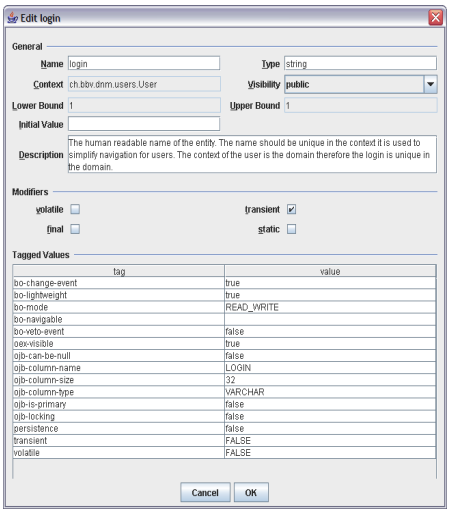
The *pmMDA* project emphasizes pragmatic use in projects and incremental progress in abstraction and power.

The tool should never require available information from the users. The users should have the possibility to overwrite build-in mechanisms for a project. In general overwriting a rule at package or class level is not supported. This approach guarantees an overall semantic in the generated application. To help the developers exhaustive semantic checks should be provided.

The building blocks used in *pmMDA* are state of the industry framework approaches

- Value object pattern
- Model view controller pattern
- Plain old object persistence
- Facade pattern
- Lightweight pattern
- Separation of business model from business logic

pmMDA Approach




```
<?xml version="1.0" encoding="ISO-8859-1"?>

...

<datatypes language="java" version="1.5">
<datatype name="bool" type="boolean"/>
<datatype name="byte" type="byte"/>
<datatype name="char" type="char"/>
<datatype name="short" type="short"/>
<datatype name="int" type="int"/>
<datatype name="long" type="long"/>
<datatype name="float" type="float"/>
<datatype name="double" type="double"/>

<datatype name="Boolean" type="java.lang.Boolean"/>
<datatype name="Byte" type="java.lang.Byte"/>
<datatype name="Char" type="java.lang.Character"/>
<datatype name="Short" type="java.lang.Short"/>
<datatype name="Integer" type="java.lang.Integer"/>
<datatype name="Long" type="java.lang.Long"/>
<datatype name="Float" type="java.lang.Float"/>
<datatype name="Double" type="java.lang.Double"/>

...

<datatype name="collection" type="java.util.Collection"
          concrete="java.util.ArrayList"/>
...
</cartridge>
```

Service Oriented Architecture

- SOA promotes programming language independence: declarative style in XML
- How to document service with complex parameters
  - WSDL *very difficult, cumbersome and not legible*
  - Programming language *no more independent*
  - MDD *how are parameter types identified*

Technically two approaches exist to develop SOA application based on web services.

- *Academical Approach*: Specify all web services using *W*eb *S*ervice *D*efinition *L*anguage – WSDL - and generates the language bindings. This approach is similar to the one used in CORBA with IDL.
- *Programmer Approach*: Code the service interface in the programming language of the server. Use tools to generate the WSDL description and the language bindings for the clients.

Th major advantage of WS against remote invocation is the independence to the programming language and the explicit separation of the interface from the service provider.

We still have an open issue to insure that the interfaces are robust. All classes used in the parameters either directly or as a member of a parameter instance must often not be part of the business model of the application. Otherwise a modification of the business model generates a unnecessary modification of the application public interfaces. Until now we have found no modeling constructs to insure this constraint in a design.

An approach could be to request that the transitive closure of classes used in a service interface are either system classes or belong to a package marked with the interface stereotype.

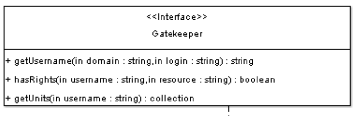
SOA Orthogonal Concepts

- Authentication, authorization, transaction, location, ...
  - Tagged value or annotations
    - Annotations supported in all environments
  - Deployment descriptors
    - Best practices in big shops
  - Aspect oriented approaches
    - The next silver bullet or the fusion of the above approaches

Similar discussions exist using other technologies to realize server applications.

pmMDA Approach

- Rule: configuration should never be in source code or model – *database name, URL, password, etc.* -
- Cartridge support deployment descriptors orthogonal to the model
  - (+) flexible
  - (-) again a new file where to add information. If you forget it, it does not work




```
struct Structure {
  char* outParam1;
  char* outParam2;
  int outParam3;
  long outParam4;
};
...
struct Array {
  int* __ptr;
  int __size;
};

enum Signals {
  OFF, ON, UNDEFINED
};

//gsoap evtv service name:      evtv01
//gsoap evtv service style:     rpc
//gsoap evtv service encoding: encoded
//gsoap evtv service namespace: http://localhost:8080/evtv01.wsdl
//gsoap evtv service location: tuxedo://127.0.0.1:1080
//gsoap evtv schema namespace:  urn:evtv01

int evtv__getValue(char* inParam1, char** result);
int evtv__setValue(char* inParam1, int inParam2, int* response);
int evtv__signal(enum Signals signal, void* response);
int evtv__getArray(struct Array* response);
int evtv__getReturnStruct(char* inParam1,
                          struct returnStructResponse* response);
```



```
<definitions name="evtv01"
 targetNamespace="http://localhost:8080/evtv01.wsdl"
 xmlns:tns="http://localhost:8080/evtv01.wsdl"
  ...
<types>
 <schema ... attributeFormDefault="unqualified">
  <import ..."/>
  <simpleType name="Signals">
   <restriction base="xsd:string">
    <enumeration value="OFF"/>
    <enumeration value="ON"/>
    <enumeration value="UNDEFINED"/>
   </restriction>
  </simpleType>
....
</types>

<message name="getValueRequest">
 <part name="inParam1" type="xsd:string"/>
</message>
  ...
<portType name="evtv01PortType">
 <operation name="getValue">
  <input message="tns:getValueRequest"/>
  <output message="tns:getValueResponse"/>
 </operation>
 ...
</portType>

<binding name="evtv01" type="tns:evtv01PortType">
 <SOAP:binding style="rpc" .../>
 <operation name="getValue">
  <SOAP:operation style="rpc" soapAction=""/>
  <input>
   <SOAP:body use="encoded" namespace="urn:evtv01"

...
<service name="evtv01">
 <port name="evtv01" binding="tns:evtv01">
  <SOAP:address location="tuxedo://127.0.0.1:1080"/>
 </port>
</service>
</definitions>
```


The next steps for the *pmMDA* are

- Finalize support of web services for Java and C++/C. The SOAP framework AXIS is used for Java, gSOAP is used both for C++ and C.
- Extends validation to detect problems in user models.
- Realize the semantic initialization of tag values when creating a new UML element or when a cartridge is added to the *pmMDA* instance at runtime.

Conclusion

- MDD is a powerful approach
  - Minimize tedious activities, simplify maintainance
  - No wizards, more complex, requires framework

- Questions & Discussion
  - MDA/MDD versus DSL / MS Factories
  - Generated versus hand-crafted code artifacts

Model driven development is a powerful approach to increase abstraction and hide platform specifics. The drawbacks are that forward engineering becomes the standard and domain knowledge is hidden in the code generators.